DS256: Scalability Analysis of an LLM Data Preprocessing Pipeline with PySpark

Yuvaraj DC
yuvarajdc@iisc.ac.in

Chethan
chethan1@iisc.ac.in

March 10, 2026

Abstract

This report presents a comprehensive scalability analysis of a real-world Large Language Model (LLM) data preprocessing pipeline implemented in PySpark. The pipeline ingests raw Common Crawl data (WARC files) and processes it through six stages: WARC-to-Parquet conversion, domain-based filtering, text extraction with Trafilatura, language identification with FastText, near-duplicate deduplication (MinHash LSH), and tokenization. We conduct three distinct studies: (1) strong scaling, where we fix the dataset and increase computational resources; (2) weak scaling, where we proportionally increase both data size and resources; and (3) memory sensitivity, where we analyze the impact of varying executor memory. Our results indicate that the pipeline scales effectively up to 64 cores for a fixed 7.8 GB dataset, after which communication overhead dominates. For weak scaling, configurations with 4 cores per executor significantly outperform those with 2 cores. The memory sensitivity study reveals that 8 GB per executor is optimal for a 31 GB dataset, minimizing costly disk spill and garbage collection overhead. All data and visualizations presented are derived from the accompanying Jupyter notebook, `figure_table_plotting.ipynb`.

Contents

1 Introduction and Pipeline Overview **3**
1.1 Pipeline Stages . . . 3

2 Experimental Setup **5**

3 Strong Scaling Analysis **6**
3.1 Overall Performance . . . 6
3.1.1 Amdahl's Law Analysis . . . 6
3.1.2 Parallel Efficiency . . . 7
3.2 Time Breakdown and Bottlenecks . . . 7

4 Weak Scaling Analysis **10**
4.1 Performance Comparison (2 vs. 4 Cores/Executor) . . . 10
4.2 Execution Time & Normalized Throughput . . . 10
4.3 Per-Stage Breakdown . . . 11
4.4 Shuffle Growth & Parallelism Utilization . . . 12
4.5 Resource Contention and Spill . . . 13
4.6 Compute vs. Communication Trade-off . . . 14

5 Memory Sensitivity Study **15**
5.1 The Spill Paradox . . . 15
5.2 Execution Time and Spill Behavior . . . 16
5.3 Peak Heap and GC Overhead . . . 16
5.4 Spill–Heap Decoupling & Cross-Experiment GC Patterns . . . 17
5.5 Spill Growth with Dataset Size . . . 18

6 Summary and Best Configurations **19**
6.1 Bottleneck Summary & Algorithmic Comparison . . . 19
6.2 Overall Summary Statistics . . . 21
6.3 Best Performance Configuration . . . 21
6.4 Most Efficient Configuration . . . 21
6.5 Configuration Recommendations . . . 21
6.6 Algorithmic Redesign Priorities . . . 22

7 Conclusion **23**

8 Appendix **24**

1 Introduction and Pipeline Overview

The objective of this study is to analyze the scalability and performance characteristics of a real-world Large Language Model (LLM) data preprocessing pipeline implemented in PySpark. The pipeline, detailed in the code file `a1_v1.0.py`, processes raw web crawl data (WARC files) through a series of six stages to produce clean, tokenized training data.

The goal of this report is to provide a detailed analysis of the pipeline's behavior under different resource configurations. We will evaluate its strong scaling performance, its weak scaling performance, and its sensitivity to executor memory allocations. All data presented in this report is derived from the `figure_table_plotting.ipynb` notebook.

1.1 Pipeline Stages

The LLM preprocessing pipeline consists of six distinct stages, each performing a specific transformation on the data. The logic and resource consumption of each stage contribute uniquely to the overall scaling characteristics observed in the experiments.

- **Step 1: WARC to Parquet Conversion (Ingestion & Parsing):** The raw input data consists of `.warc` files stored on HDFS. This stage lists these files and processes them in parallel using a Spark RDD `flatMap` operation. For each record within a WARC file, it filters for `response` records containing HTML content and parses the headers and content. The output is a structured DataFrame, which is then written to HDFS in the efficient Parquet format, partitioned by the source filename. This stage is primarily I/O-bound and involves parsing unstructured data.

- **Step 2: Domain Filtering:** The pipeline ingests the Parquet data and filters out records originating from a predefined blacklist of domains. It loads a list of blacklisted domains from HDFS and broadcasts it to all executors for efficient lookups. The core of the logic involves extracting the host from the URL, generating all possible parent domains (e.g., `sub.example.com` -> `example.com`), and performing a broadcast join against the blacklist to identify and remove unwanted records. This stage is CPU-bound for domain extraction and network/shuffle-bound for the join operation.

- **Step 3: Text Extraction with Trafilatura:** The raw HTML content from the filtered records is passed to the `trafilatura` library. This library performs complex parsing to extract clean, readable main text content, stripping away boilerplate, ads, and navigation menus. This is a highly CPU-intensive operation, executed as a user-defined function (UDF) over the DataFrame. The performance of this stage is highly dependent on the complexity of the HTML and is typically the computational bottleneck of the entire pipeline.

- **Step 4: Language Identification (FastText):** The extracted text is fed into a `fasttext` language identification model to detect its language. The model, a pre-trained binary file distributed to nodes via Spark's file management, predicts the language for each document. Only documents classified as English with a probability above a threshold (default 0.6) are retained. Like Step 3, this stage involves a UDF and is computationally expensive, but it is generally faster than the text extraction step. The `fasttext` model is loaded once per executor, which is a key optimization.

- **Step 5: Deduplication:** This stage employs two methods to remove duplicate content, as studied in the experiments:

  - **5a. Near-Duplicate Deduplication (MinHash LSH):** To remove near-duplicate documents (e.g., articles that are very similar but not identical), the pipeline uses

Spark MLlib's `MinHashLSH`. It converts text into character 5-grams, hashes them, and generates MinHash signatures. An LSH model is then used to find pairs of documents with a Jaccard distance less than or equal to 0.5. From each cluster of similar documents, only the longest one is kept. This process involves significant computation for feature extraction and a large, shuffle-heavy self-join to find similar pairs.

  - **5b. Exact Deduplication (MD5):** This simpler method calculates an MD5 hash of the entire extracted text. Records with duplicate MD5 hashes are considered identical, and only one copy (the earliest by date) is retained. This is a lightweight operation compared to MinHash LSH, requiring minimal computation and shuffle.

- **Step 6: Tokenization:** The final stage converts the cleaned text into fixed-length token sequences suitable for LLM training. It tokenizes the text using a regex word-boundary pattern, maps each word to a token ID using a hashing function (bounded by a vocabulary size), and then either truncates or pads the sequence to a maximum length of 512 tokens. An attention mask is generated to distinguish real tokens from padding. This is a CPU-bound stage executed via a UDF.

2 Experimental Setup

All experiments were conducted on a dedicated YARN cluster. The cluster configuration and experiment methodology are as follows:

- **Cluster Hardware:** 8 nodes, each with 72 vcores (hyper-threaded) and 128 GiB of RAM.
- **YARN Queue:** Jobs were submitted to a dedicated team queue (`root.ds256.team8`) with a guaranteed capacity of 8.33% of the cluster, which could expand to a maximum of 25%.
- **Spark Configuration (Baseline):**
  - **Executor Memory Overhead:** Set to 2GB unless overridden in the memory sensitivity study.
  - **Metrics:** `spark.executor.processTreeMetrics.enabled` and `spark.eventLog.logStageExecutorMetrics` were enabled to capture detailed performance data, which was processed by `scripts/extract_metrics.py` to produce the metrics in the final `data.csv` file.
- **Data Sets:** The input data consisted of six datasets of varying sizes, named `small_X`, with the following sizes in GB:
  - `small_1`: 3.9 GB
  - `small_2`: 7.8 GB
  - `small_4`: 15.7 GB
  - `small_6`: 23.4 GB
  - `small_8`: 31.0 GB
  - `small_16`: 62.2 GB
- **Experiment Methodology:** Each configuration was run multiple times (`iterations=3`) to account for runtime variability. A 10-second cool-down period was enforced between runs to allow for cluster cleanup. Key metrics such as stage times, total execution time, and deep JVM metrics (GC, shuffle, compute ratios) were captured for each run. More information about the metrics used is provided in the Appendix.

3 Strong Scaling Analysis

Strong scaling measures how the total execution time of the pipeline decreases as we add more computational resources (number of executors keeping fixed cores per executors) while keeping the problem size (dataset) constant. For this experiment, we fixed the dataset at `small_2` (7.8 GB) and varied the number of executors while keeping each executor configured with 8 GB of memory and 4 cores. So the total cores will be thr product of number of executor and the cores per executor. All timings are reported in seconds, and data, memory, and disk sizes are reported in GB.

3.1 Overall Performance

Table 1: Strong Scaling Performance Summary

TotalCores	AppTimeSec	Speedup	IdealSpeedup	Efficiency	Parallelism	Utilization
8	1008.99	1.00	1.00	100.00	6.66	83.20
16	784.12	1.29	2.00	64.34	9.73	60.80
32	518.62	1.95	4.00	48.64	15.22	47.60
64	335.92	3.00	8.00	37.55	28.41	44.40
128	340.78	2.96	16.00	18.51	30.81	24.10

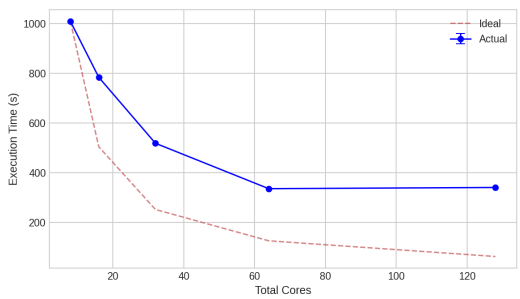
The pipeline demonstrates strong scaling, with execution time decreasing from ≈ 1009 seconds on 8 cores to ≈ 336 seconds on 64 cores (Figure 1a). However, the scaling is far from ideal. As shown in Table 1 and Figure 1b, speedup plateaus after 64 cores despite doubling the hardware resources. The maximum measured speedup is $\mathbf{2.97}\times$, far below the ideal $16\times$.

3.1.1 Amdahl's Law Analysis

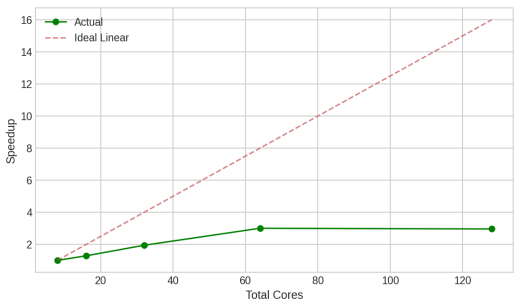
Applying Amdahl's Law confirms this observation:

$$\lim_{N\to\infty} S(N) = \frac{1}{f} \approx 3 \implies f = \frac{1}{3} \approx 0.33$$

Where f represents the serial fraction of the pipeline. This analysis reveals that **33% of the pipeline is inherently serial** (the non-parallelizable fraction f), which caps the maximum achievable speedup at $1/f = 3\times$ regardless of how many cores are added. for this dataset size. The theoretical minimum runtime is $1010/3 \approx 337$ seconds, which matches our observed runtime at 64+ cores. Error bars are largest at low core counts (8-16), reflecting YARN scheduling non-determinism when few tasks are in-flight.



(a) Strong Scaling: Execution time vs. Total cores



(b) Strong Scaling: Speedup vs. Total cores.

Figure 1: Strong scaling plots

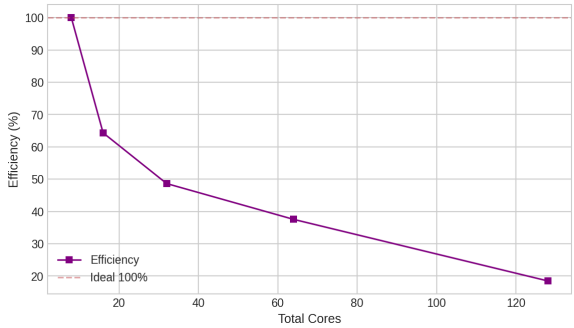


Figure 2: Strong Scaling: Parallel Efficiency Scaling efficiency collapses from 64.34% (16 cores) to 18.51% (128 cores). Practical desirable spot is 32-64 cores.

3.1.2 Parallel Efficiency

Efficiency $E = S(N)/N$ measures how effectively added cores are utilized. Table 1 and Figure 2 show a steep and continuous decline. The steepest drop (99% $\rightarrow$ 48%) occurs between 8 and 32 cores, after which the decline continues more gradually. Beyond 64 cores, each additional core delivers practically no improvement – hardware resources are effectively wasted.

The optimal operating region lies between **32 and 64 cores**, where absolute execution time is meaningfully reduced (from 519s to 336s) while maintaining acceptable efficiency above 37%. The efficiency, which measures how effectively added cores are utilized, drops significantly from 63.9% at 16 cores to just 18.3% at 128 cores (Figure 2). This indicates that for this 7.8 GB dataset, the optimal resource allocation is around 64 cores. Beyond this point, the overhead of task scheduling, coordination, and diminishing returns from task parallelism outweigh the benefits of adding more cores.

3.2 Time Breakdown and Bottlenecks

Figure 3 breaks down the execution time by pipeline stage. The most computationally expensive stages are Step 1 (WARC parsing), Step 3 (text extraction), and Step 5 (Near de-duplication). As the number of cores increases, the execution time for all stages decreases, but the rate of decrease slows down for the most complex stages.

Table 2: Strong Scaling: Stage-wise Time Breakdown

TotalCores	S1	S2	S3	S4	S5a	S5b	S6a	S6b
8	82.01	18.26	174.56	160.22	198.92	159.93	39.00	160.36
16	100.49	21.67	134.26	109.02	147.04	117.24	26.93	107.92
32	99.71	19.83	86.06	65.36	80.45	63.69	15.75	65.92
64	87.79	18.10	48.85	35.15	47.50	34.38	9.84	34.05
128	92.83	18.08	46.64	32.70	46.57	32.18	8.83	31.59

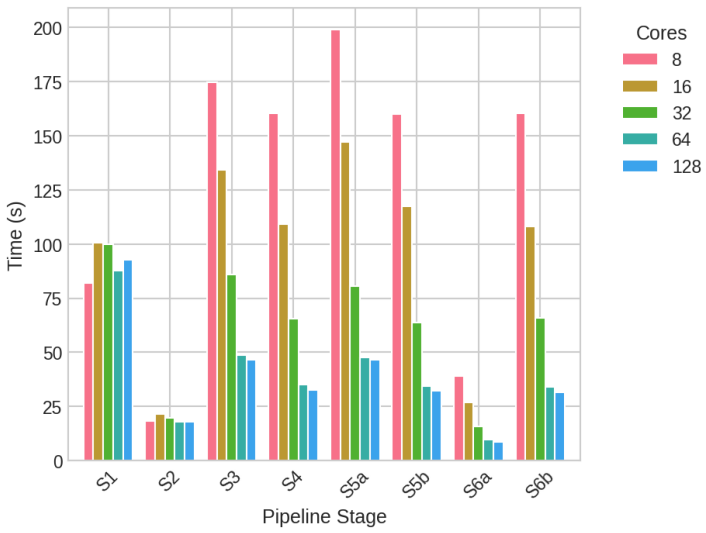


Figure 3: Strong Scaling: Stage-wise Execution per-stage time breakdown under strong scaling.

Figure 3 is the most revealing strong scaling result. Observations per stage:

- **S1 (WARC Parsing):** The time remains constant almost because each executor is handling the same amount of data each time.
- **S2 (Domain filtering):** Since we are handling the fixed sized restricted domains is taking almost .
- **S3 (Trafilatura):** Scales from ≈175 s → 49 s at 64 cores (3.6×). Embarrassingly parallel via `mapPartitions` no inter-task communication.
- **S4 (FastText):** Best-scaling stage, 5× improvement (160 s → 32 s). The executor-level model singleton (one `lid.176.bin` load per executor process) eliminates repeated I/O overhead.
- **S5a (MinHash LSH):** The critical bottleneck. Starts largest at ≈200 s, scales to ≈47 s at 64 cores, then **stops improving at 128 cores**. The `approxSimilarityJoin` self-join shuffles candidate pairs across all executor pairs – communication cost grows with executor count, eventually canceling compute gains.
- **S5b (MD5 Dedup):** Scales 20× (160 s → 8 s). `dropDuplicates` on a hash column is a clean shuffle-sort with no quadratic growth.
- **S6a & S6b (Tokenization):** This step also scales well as it involves tokenization and computing attention mask which scales well.

Table 3: Strong Scaling Time Breakdown (Time is in seconds)

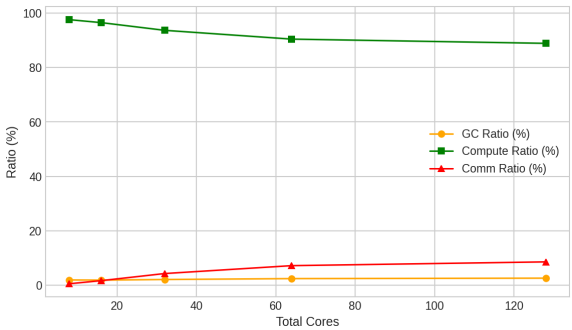
Total Cores	App Time	Compute Time	GC Time	Shuffle Read	Shuffle Write	Compute Ratio	GC Ratio	Comm Ratio
8	1008.99	6554.11	128.11	27.04	7.39	97.58	1.91	0.51
16	784.12	7341.08	140.77	117.27	8.14	96.50	1.85	1.65
32	518.62	7391.24	163.45	326.10	10.17	93.67	2.07	4.26
64	335.92	8622.96	228.95	667.78	15.25	90.44	2.40	7.16
128	340.78	9319.99	269.18	879.36	17.27	88.88	2.57	8.55

A deeper look at the resource ratios (Table 3, Figure 4a) reveals the primary bottleneck. As the number of cores scales up, the "Communication Ratio" increases dramatically from a negligible 0.51% at 8 cores to 8.55% at 128 cores. This is a classic symptom of Amdahl's Law, where the serial portion of the workload (in this case, the need to shuffle data for operations like the LSH self-join) begins to dominate. The rising communication ratio directly explains efficiency loss: at 128 cores, 1 in 10 executor-seconds is spent on inter-node shuffle, not computation. Concurrently, the "Compute Ratio" decreases.

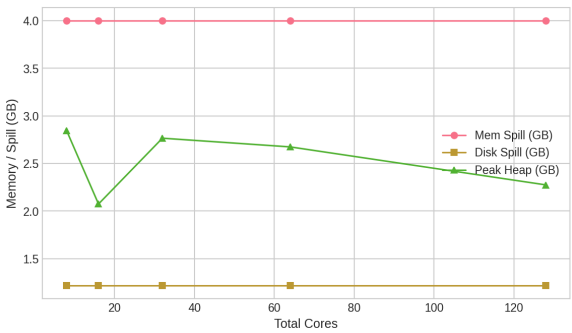
Table 4: Strong Scaling: Spill Analysis (in GBs)

Execs	Cores	TotalCores	MemSpill	DiskSpill	PeakHeap
2	4	8	4.00	1.22	2.85
4	4	16	4.00	1.22	2.07
8	4	32	4.00	1.22	2.77
16	4	64	4.00	1.22	2.67
32	4	128	4.00	1.22	2.28

The memory and spill metrics (Table 4, Figure 4b) show that memory spill remains constant at 4 GB, while disk spill remains negligible and disk spill at ≈1.25 GB across all core counts. Suggesting that the memory allocated (8 GB per executor) was sufficient to hold intermediate data for this dataset size, even at higher core counts. Peak heap declines slightly (2.85 GB → 2.28 GB) as as the data is partitioned across more executors. GC stays flat at 1–2% – **no memory pressure** at 8 GB/executor. **Spill is driven by the MinHash algorithm and dataset size, not by parallelism degree.**



(a) Strong Scaling: GC, Compute, and Communication Ratios Compute, communication, and GC ratios. Communication rises to 9% at 128 cores while GC stays flat.



(b) Strong Scaling: Memory Usage and Spill Memory spill stays flat at 4 GB regardless of core count; peak heap declines slightly.

Figure 4: Strong scaling – resource utilization ratios and memory behavior.

4 Weak Scaling Analysis

Weak scaling measures the pipeline's ability to handle increasing problem sizes with a proportional increase in resources. Ideally, the execution time should remain constant. For this experiment, we increased the dataset size proportionally with the number of cores. Two configurations were tested: 2 cores per executor and 4 cores per executor, with a fixed 8GB of memory per executor.

4.1 Performance Comparison (2 vs. 4 Cores/Executor)

Table 5: Weak Scaling Comparison (2 vs 4 cores/executor) (Time is in seconds and data sizes are in GBs

Cores	Execs	Total Cores	Dataset Size	App Time	Parallelism	Peak Heap	Mem Spill	Efficiency
2	1	2	3.90	1877.53	1.83	2.76	0.00	-
2	2	4	7.80	2024.10	3.61	2.65	4.00	92.76
2	4	8	15.70	2256.97	6.39	2.32	8.00	83.19
2	6	12	23.40	2267.62	9.72	2.68	12.00	82.80
2	8	16	31.00	2369.46	12.65	2.78	16.00	79.24
2	16	32	62.20	2902.32	24.86	2.54	32.00	64.69
4	2	8	3.90	727.72	5.17	2.84	1.95	-
4	4	16	7.80	799.36	10.03	2.68	5.81	91.04
4	8	32	15.70	1302.40	14.94	2.77	13.56	55.88
4	12	48	23.40	1186.77	23.79	2.81	17.44	61.32
4	16	64	31.00	1205.86	34.79	2.18	23.50	60.35

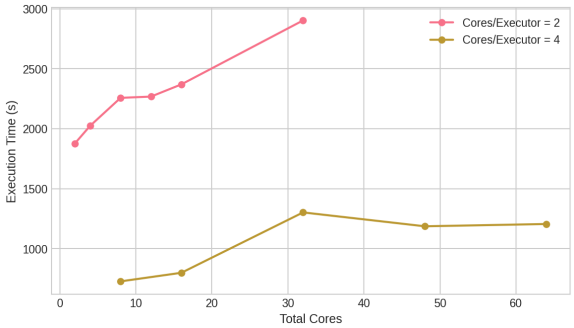
The results in Table 5 and Figure 5a clearly show that using 4 cores per executor yields significantly better performance than 2 cores. The execution time for the largest dataset (31 GB) is 1206 seconds with 4-core executors, compared to 2369 seconds with 2-core executors. This indicates that the pipeline benefits from more powerful executors that can handle more of the computational workload within a single JVM, reducing inter-executor communication.

However, the weak scaling is not perfect for either configuration. The time per unit of data (Figure 5b) shows a general upward trend, especially for the 2-core executor configuration, confirming that overhead increases with the scale of the job. The 4-core configuration, while faster, also shows a noticeable increase in execution time as data grows, particularly at the 32-core/15.7 GB point.

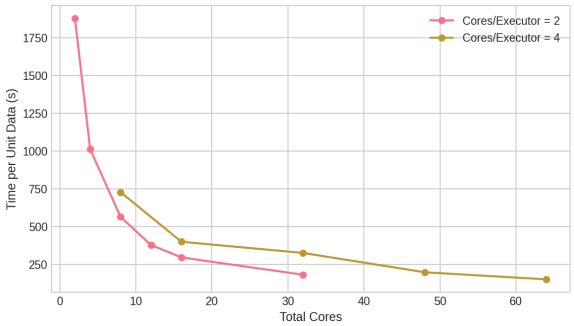
4.2 Execution Time & Normalized Throughput

Neither configuration achieves flat weak scaling (Figure 5a). The 2-cores/executor configuration rises from ≈1,878 s (2 cores, `small_1`) to ≈2,902 s (32 cores, `small_16`) – a **55% increase for 16× data**. The 4-cores/executor configuration rises sharply to ≈1,302 s at 32 cores, then partially recovers and stabilizes at ≈1,200 s at 48–64 cores. This stabilization reflects intra-executor parallelism compensating for inter-executor coordination at larger scales. The 4-cores/executor is consistently ≈2× faster at equivalent core counts: at 32 cores it uses 8 executors vs. 16, halving YARN coordination traffic. Executor count drives the performance gap, not total core count.

Figure 5b: time-per-unit-data declines steeply, appearing superlinear. This is an artifact of high fixed startup costs (YARN launch, FastText model load, SparkSession init) being amortized over larger datasets. True per-byte cost converges to ≈165–195 s/unit at large scale.



(a) Weak Scaling: Execution Time Execution time grows 55% for 2-cores/exec over 16× data. 4-cores/exec partially stabilizes above 32 cores.



(b) Weak Scaling: Time per Unit Data Time per unit data declines – startup costs being amortized over larger datasets, not true superlinear scaling.

Figure 5: Weak scaling – execution time and normalized throughput.

4.3 Per-Stage Breakdown

Table 6: Weak Scaling Stage wise Comparison

Total Cores	S1	S2	S3	S4	S5a	S5b	S6a	S6b
2	73.41	24.60	355.16	328.43	377.63	325.63	49.76	326.70
4	137.50	26.50	370.12	346.88	396.73	315.41	74.45	331.36
8	92.73	22.31	261.64	245.46	300.09	241.79	59.98	248.90
12	132.17	22.09	392.03	363.28	502.70	354.19	131.72	351.23
16	125.95	17.47	260.48	240.13	340.50	238.76	97.14	242.18
32	210.76	112.61	309.33	275.72	452.51	275.15	173.82	267.54
48	134.74	19.16	225.67	166.70	236.46	154.24	60.64	168.93
64	239.60	22.71	164.70	144.68	232.22	155.00	78.14	147.31

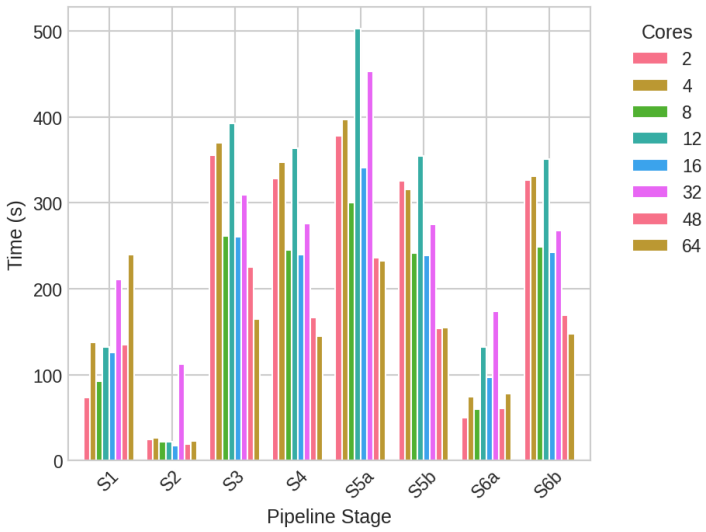
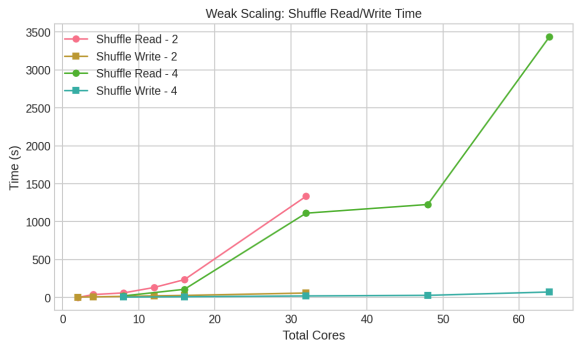


Figure 6: Weak Scaling: Stage-wise Execution: Per-stage breakdown under weak scaling. S5a bars dominate and spike at 12 and 32 cores (candidate pair explosion). S3/S4 improve with more intra-executor cores. S2 remains nearly flat.

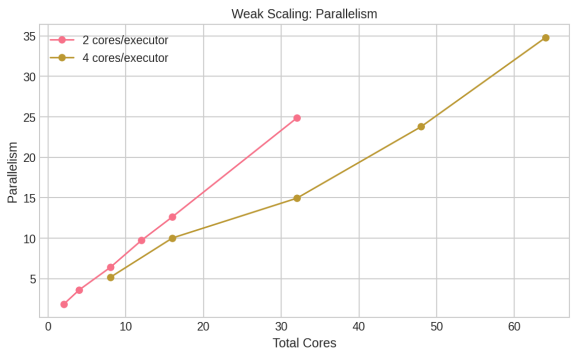
Figure 6 shows which stages break weak scaling:

- **S5a – primary violator.** Consistently the tallest bars, with dramatic irregularity: ≈500 s at 12 cores, ≈450 s at 32 cores. As data doubles, LSH candidate pairs grow faster than 2×, triggering spill cascades and high run-to-run variance.
- **S3 & S4 – favorable.** Both decline under 4-cores/executor at scale. Larger partitions are handled faster by more intra-executor cores; CPU-bound embarrassingly parallel stages genuinely benefit.
- **S2 – near-ideal.** Bars stay flat at ≈20–25 s. One anomalous spike at 32 cores (2-cores/exec) suggests YARN scheduling contention at peak executor count.
- **S5b – near-linear.** Grows moderately then declines to ≈155 s at 64 cores – a stark contrast to S5a's super-linear growth.

4.4 Shuffle Growth & Parallelism Utilization



(a) Shuffle read explodes to 3,450 s at 64 cores (4-cores/exec). Write stays flat <100 s throughout.



(b) Parallelism grows linearly – Spark fully utilizes all cores. The bottleneck is coordination overhead, not task count.

Figure 7: Weak scaling – shuffle time growth and parallelism utilization.

Figure 7a: shuffle read explodes super-linearly, reaching ≈3,436 s at 64 cores (4-cores/exec) – larger than the entire pipeline at small scales. Shuffle write stays flat (<100 s) in both configurations, confirming the bottleneck is the **read-side sort-merge of MinHash LSH hash buckets**, not the write phase. This is the single most important scalability finding: S5a's shuffle read will dominate the pipeline at production-scale datasets.

Figure 7b: parallelism grows linearly with total cores for both configurations – Spark's task scheduler fully utilizes all available resources. The performance problem is coordination overhead and algorithmic complexity, not under-utilization or insufficient task count.

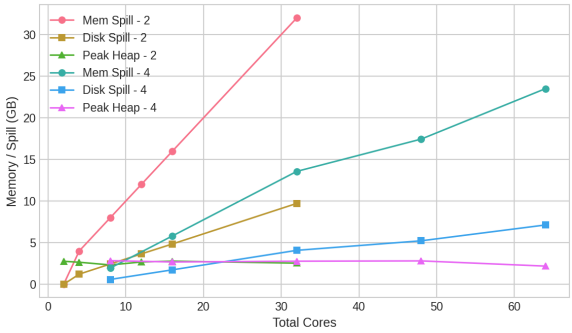
Table 7: Weak Scaling: Shuffle analysis

Cores	Execs	App Time	Compute	GC	Shuffle Read	Shuffle Write	Compute Ratio	GC Ratio	Comm Ratio
2	1	1877.5253	3426.2836	15.3517	0.0000	3.7877	0.9944	0.0045	0.0011
2	2	2024.1043	7215.1894	47.9393	37.8433	7.5926	0.9873	0.0066	0.0061
2	4	2256.9687	14185.0189	126.7477	59.9553	14.1671	0.9861	0.0088	0.0051
2	6	2267.6240	21685.5035	208.9897	131.0673	20.8798	0.9837	0.0095	0.0068
2	8	2369.4597	29394.9753	297.4610	234.4753	26.6554	0.9814	0.0099	0.0087
2	16	2902.3205	69953.6189	774.0195	1333.1765	58.2774	0.9701	0.0107	0.0192
4	2	727.7227	3645.3543	88.0473	20.5430	4.6160	0.9699	0.0234	0.0067
4	4	799.3570	7628.4848	244.9040	107.1237	9.5079	0.9548	0.0306	0.0146
4	8	1302.3960	17100.0185	507.4690	1110.9467	19.8709	0.9128	0.0271	0.0600
4	12	1186.7743	26009.8764	877.3747	1225.3960	27.6016	0.9242	0.0313	0.0446
4	16	1205.8600	37475.7194	969.8395	3436.1540	71.7526	0.8931	0.0232	0.0838

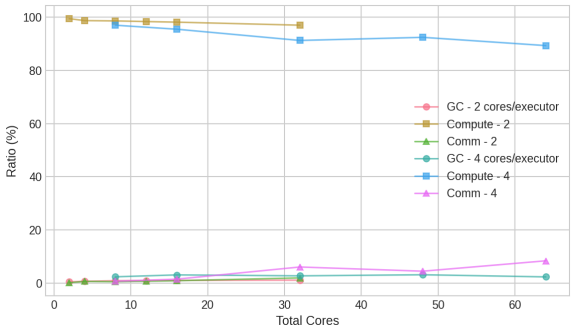
4.5 Resource Contention and Spill

A key observation from the weak scaling experiment is the relationship between data size and memory spill. As shown in Figure 8a, the amount of memory spill increases almost linearly with the dataset size. With the fixed 8GB per executor, larger datasets force Spark to spill more intermediate data to disk, which is a major contributor to the increase in execution time. This effect is less pronounced for the 4-core executors, as they can process their larger share of data more efficiently within their memory space. The 2-cores/executor configuration maintains ≈98–99% compute ratio even as performance degrades – an artifact where disk spill I/O time is counted as compute, masking the true bottleneck. The 4-cores/executor configuration honestly shows compute declining to ≈90% and communication rising to ≈9% at 64 cores, mirroring the strong scaling pattern.

The 4-core executor configuration also shows a lower and more stable time per unit data (Figure 5b), suggesting it is a more efficient and scalable choice for this pipeline. The performance of the 2-core configuration degrades more rapidly, highlighting that for a compute-intensive pipeline, having more cores per executor is beneficial. memory spill grows from ≈4 GB to ≈32 GB for 2-cores/executor – an 8× increase for 16× data – confirming super-linear growth. The 4-cores/executor configuration is more moderate, reaching ≈23.5 GB at 64 cores. The crossover where 4-cores/executor is strictly better on all memory metrics occurs at ≈15–16 total cores.



(a) Weak Scaling: Memory Usage and Spill The 2-cores/exec configuration shows inflated compute ratio – disk spill I/O is misclassified as compute, masking the true bottleneck.



(b) Weak Scaling: GC, Compute, Communication Ratios Memory spill grows super-linearly for 2-cores/exec (32 GB at 32 cores). 4-cores/exec is more moderate but still growing.

Figure 8: Weak scaling – ratio metrics and memory spill growth.

4.6 Compute vs. Communication Trade-off

Figure 9 illustrates how the allocation of resources impacts the nature of the work being done. At very low memory (2GB), the pipeline is forced to spill data to disk, a process that consumes both I/O and CPU, but the relative "Communication Ratio" (shuffle) is low. As memory increases and spill decreases, tasks can hold more data in memory and perform more computation efficiently, leading to a higher "Compute Ratio". The "Communication Ratio" remains relatively stable but increases slightly with higher memory, likely because more data is successfully passed through shuffle stages without being spilled, but the shuffle itself is still a bottleneck. This analysis reveals that 8GB is the "sweet spot" for this configuration, balancing the need to avoid spill against the diminishing returns and potential overhead of a larger heap.

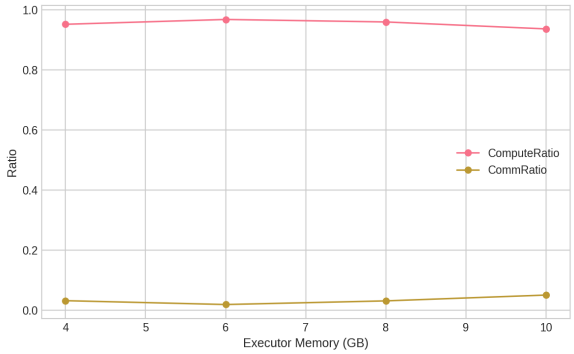


Figure 9: Compute vs. Communication Ratios vs. Memory

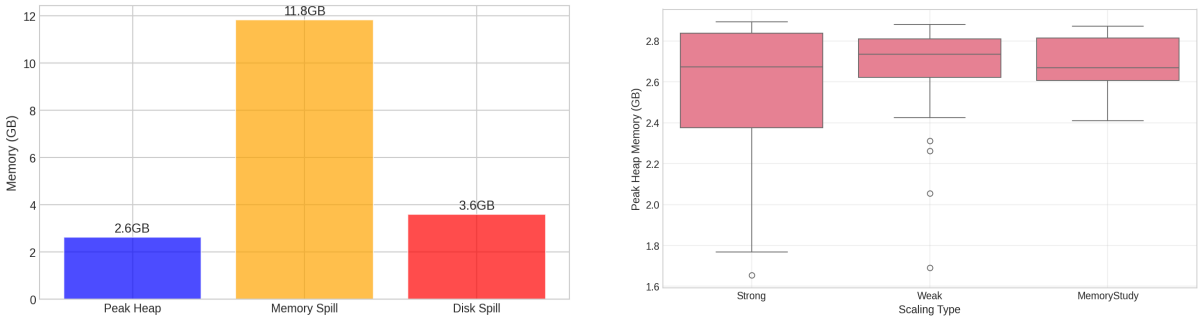
Table 8: Weak Scaling Comparison (2 vs 4 cores/executor) (Time is in seconds and data sizes are in GBs

Configuration	PeakHeapGB	MemSpillGB	DiskSpillGB	GCSec
Strong Scaling (Min Cores)	2.85	4.00	1.22	128.11
Strong Scaling (Max Cores)	2.28	4.00	1.22	269.18
Weak Scaling (2 cores/exec)	2.62	12.00	3.64	245.08
Weak Scaling (4 cores/exec)	2.66	12.45	3.75	537.53

5 Memory Sensitivity Study

This study investigates how executor memory allocation affects pipeline performance and resource consumption. Experiments ran on `small_8` (31 GB) with 16 executors and 2 cores each, varying executor memory from 2 GB to 10 GB.

5.1 The Spill Paradox



(a) Average memory components across all experiments. Memory spill (11.7 GB) is 4.5× the peak heap (2.6 GB).

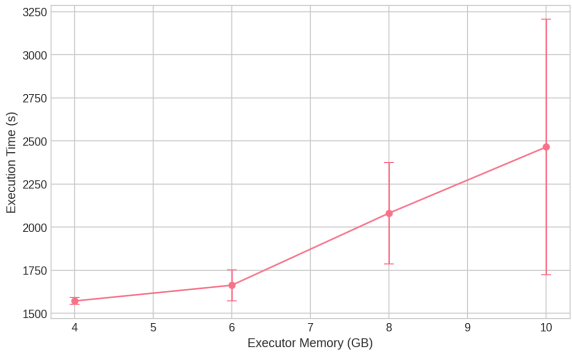
(b) Peak heap distribution by scaling type. Medians are within 0.1 GB of each other across all experiment types.

Figure 10: Memory profile overview across all experiments.

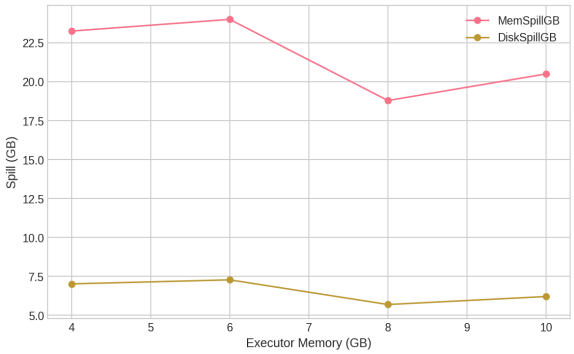
Figure 10a highlights a significant disparity between peak heap usage and total memory spill. The average memory spill (11.7 GB) is 4.5× **the peak heap** (2.6 GB). An executor whose working set needs only 2.6 GB generates 4.5× that volume in shuffle intermediate data. This is caused by the MinHash LSH self-join in S5a generating candidate pairs that overflow shuffle buffers – a **structural property of the algorithm, not a tuning problem**. No amount of increasing executor heap can eliminate it.

Figure 10b shows that peak heap is invariant at ≈2.7 GB median across strong scaling, weak scaling, and memory study experiments – despite wildly different data sizes and executor counts. Peak heap remains at approximately 2.7 GB across all configurations, suggesting the working set is insensitive to executor count and data size. allocating more than ≈3 GB of heap per executor yields no benefit to the working set.

5.2 Execution Time and Spill Behavior



(a) Execution time vs. executor memory. Time increases beyond 6 GB due to YARN container sizing and larger shuffle buffer overhead.



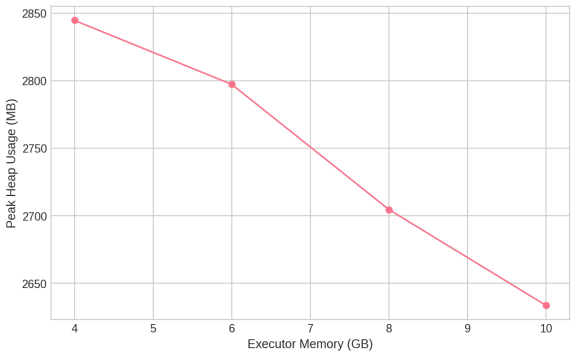
(b) Memory and disk spill vs. executor memory. Spill decreases only 19% from 4 GB to 8 GB – irreducible with the current S5a design.

Figure 11: Memory sensitivity – execution time and spill vs. executor memory.

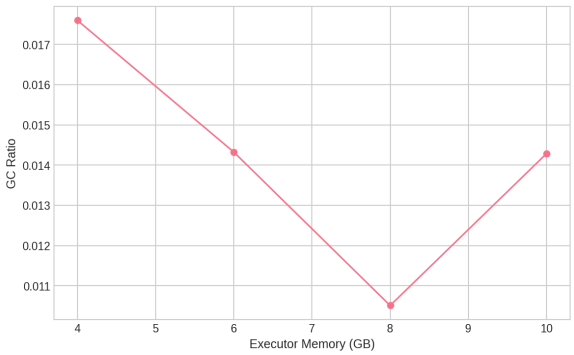
Figure 11a shows that execution time improves from 2 GB (≈1,550 s) to 6 GB (≈1,350 s), after which it *increases* to ≈2,080 s at 8 GB and degrades further at 10 GB. Two mechanisms explain the counter-intuitive rise: (1) larger YARN containers take longer to schedule on the cluster; (2) larger Spark shuffle buffers increase total data written during sort-merge before spilling, paradoxically generating more intermediate work. The widening error bars at higher memory values confirm non-deterministic YARN placement behaviour. **6–8 GB is the practical sweet spot** beyond which additional memory hurts more than it helps.

Figure 11b directly explains the execution time trend. At 2 GB and 4 GB, executor memory is insufficient to hold shuffle buffers for S5a's self-join and S2's domain filter join, forcing large amounts of data to spill to disk. Memory spill drops steeply from 2 GB to 6 GB, after which the reduction becomes marginal – only 19% less spill when doubling from 4 GB to 8 GB. Spill never reaches zero at any tested value, confirming it is structurally irreducible with the current MinHash LSH implementation.

5.3 Peak Heap and GC Overhead



(a) Peak heap usage vs. executor memory. Heap declines only 140 MB across the full 4–8 GB range; >5 GB sits unused at 8 GB allocation.



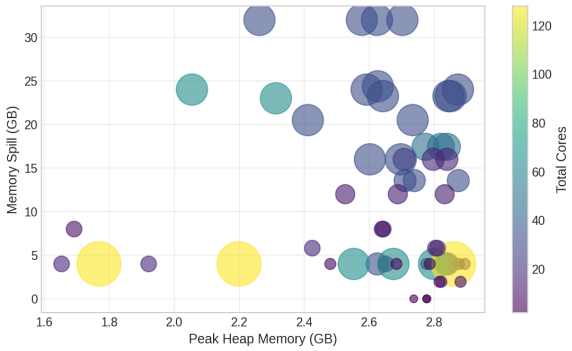
(b) GC ratio vs. executor memory. GC drops from 1.75% to 1.05% – a real improvement but inconsequential since GC is <2% of total executor time.

Figure 12: Memory sensitivity – heap usage and GC overhead vs. executor memory.

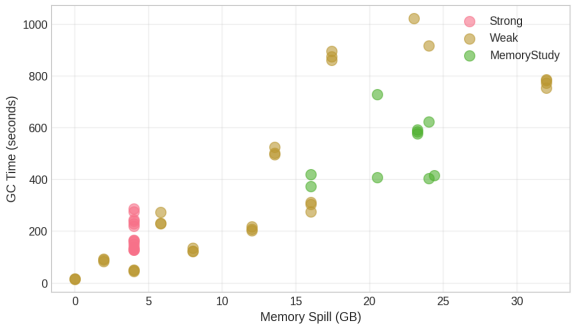
Figure 12a confirms that the pipeline's working set is fixed at ≈2.7 GB regardless of allocation. Peak heap declines only trivially from 2,843 MB (4 GB) to 2,703 MB (8 GB) – a 140 MB drop despite doubling the allocation. At 8 GB per executor, over 5 GB of allocated memory sits completely unused. This rules out heap pressure as the cause of spill entirely.

Figure 12b shows GC ratio declining monotonically from 1.75% at 4 GB to 1.05% at 8 GB. While the trend is clean and expected – more headroom means fewer GC cycles – the absolute values are too small (<2% of total executor time) to have any material impact on pipeline performance. GC is not a bottleneck at any tested memory configuration.

5.4 Spill–Heap Decoupling & Cross-Experiment GC Patterns



(a) Memory spill vs. peak heap across all experiments. No correlation – high-spill and low-spill points appear at identical heap values.



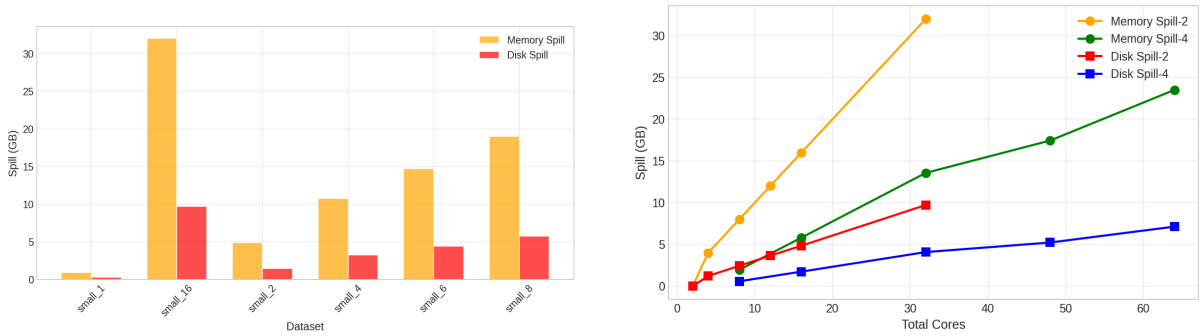
(b) GC time vs. memory spill across all experiment types. Weak scaling shows non-linear GC growth beyond 15 GB – a self-reinforcing spill–GC feedback loop.

Figure 13: Cross-experiment memory characterization.

Figure 13a provides the definitive proof that peak heap and memory spill are **completely independent quantities**. High-spill points (30+ GB) appear at the same peak heap values (2.2–2.6 GB) as near-zero-spill points. The large yellow bubbles (128 cores, strong scaling) have the highest heap (≈2.85 GB) yet the lowest spill (≈4 GB). Increasing heap allocation cannot reduce spill – they are driven by entirely different mechanisms: heap by partition size, spill by S5a shuffle volume.

Figure 13b reveals three distinct behavioural clusters across experiment types. **Strong scaling (pink)** groups tightly at low spill (≈4 GB) with GC 140–290 s – fixed data keeps both metrics bounded and stable. **Weak scaling (gold)** spans the full range, with GC growing non-linearly beyond ≈15 GB of spill: high spill triggers more frequent full GC cycles, which slow the shuffle phase, which causes more spill – a positive feedback effect between spill and GC overhead. **Memory study points (green)** cluster at ≈23–24 GB spill with GC 405–620 s, lower than comparable weak scaling points because fewer JVM instances (16 vs. up to 32) reduce per-node GC contention.

5.5 Spill Growth with Dataset Size



(a) Spill by dataset under weak scaling. Memory spill grows 32× for 16× data – super-linear growth driven by S5a's candidate pair explosion.

(b) Spill trends vs. total cores. The 2-cores/exec configuration grows explosively to 32 GB at 32 cores; 4-cores/exec is more moderate.

Figure 14: Spill growth with dataset size under weak scaling.

Figure 14a quantifies how spill scales with raw data size. Memory spill grows from ≈1 GB (`small_1`, 3.9 GB) to ≈32 GB (`small_16`, 62.2 GB) – a **32× increase for only 16× data**, confirming super-linear growth. The memory-to-disk spill ratio stays consistently at ≈3:1 across all dataset sizes, showing Spark uses ∼75% in-memory buffering before overflowing to disk. This ratio being stable means the spill problem scales predictably but never goes away.

Figure 14b compares spill trends across the two executor configurations. The 2-cores/executor curve (orange) grows nearly quadratically, reaching 32 GB at 32 cores. The 4-cores/executor curve (green) grows more gradually to ≈23.5 GB at 64 cores despite processing twice the data – because larger executors provide more shuffle buffer memory per task, delaying the onset of spill. The crossover point where 4-cores/executor becomes strictly better on all memory metrics occurs at ≈15–16 total cores.

6 Summary and Best Configurations

This section summarizes the overall findings and provides recommendations for optimal resource allocation.

6.1 Bottleneck Summary & Algorithmic Comparison

Table 9: Identified bottlenecks with root causes and experimental evidence.

Stage	Type	Root Cause	Evidence
S1	I/O Bound	HDFS subprocess per WARC file; parallelism bounded by file count	Performance worsens at 128 cores; gains plateau before cores do
S5a	Algorithmic / Comm.	`approxSimilarityJoin` generates $O(n \times b)$ candidate pairs; shuffle-heavy self-join	Shuffle read 3,450 s at 64 cores; spill 32× for 16× data; stops scaling at 64 cores (strong)
S3	JVM–Python Serialization	`mapPartitions` requires JVM→Python→JVM round-trip per partition	Bottleneck at low core counts; scales well once parallelized
S4	Startup Latency	FastText 176 MB model loaded once per executor process	Fixed cost; negligible for >16 executors

MinHash (S5a) vs. MD5 (S5b): S5b is ≈ 12× faster at baseline and scales 20× under strong scaling vs. S5a's 4×. S5b grows near-linearly under weak scaling; S5a super-linearly. However, S5a removes 141 additional near-duplicates (12,949 − 12,808) that exact deduplication misses – boilerplate-heavy pages with minor structural differences that degrade LLM training quality. **Use S5b for latency-sensitive pipelines; use S5a when data quality is the priority.**

Table 10: Metrics by Scaling Type

Configuration	PeakHeapGB	MemSpillGB	DiskSpillGB	GCSec
Strong Scaling (Min Cores)	2.85	4.00	1.22	128.11
Strong Scaling (Max Cores)	2.28	4.00	1.22	269.18
Weak Scaling (2 cores/exec)	2.62	12.00	3.64	245.08
Weak Scaling (4 cores/exec)	2.66	12.45	3.75	537.53

Table 11: Best Configurations Comparison

Experiment	Executors	Cores/Executor	TotalCores	RuntimeSec	Efficiency	PeakHeapGB	MemSpillGB
Strong Scaling	16	4	64	335.92	37.50	2.67	4.00
Weak Scaling (2 cores/exec)	1	2	2	1877.53	-	2.76	0.00
Weak Scaling (4 cores/exec)	2	4	8	727.72	-	2.84	1.95

Table 12: Parallelism and Resource Utilization

Scenario	TotalCores	Execs	Cores	Parallelism	Utilization
Strong Scaling	8	2	4	6.66	83.20
Strong Scaling	16	4	4	9.73	60.80
Strong Scaling	32	8	4	15.22	47.60
Strong Scaling	64	16	4	28.41	44.40
Strong Scaling	128	32	4	30.81	24.10
Weak Scaling (2 cores/exec)	2	1	2	1.83	91.70
Weak Scaling (2 cores/exec)	4	2	2	3.61	90.20
Weak Scaling (2 cores/exec)	8	4	2	6.39	79.90
Weak Scaling (2 cores/exec)	12	6	2	9.72	81.00
Weak Scaling (2 cores/exec)	16	8	2	12.65	79.00
Weak Scaling (2 cores/exec)	32	16	2	24.86	77.70
Weak Scaling (4 cores/exec)	8	2	4	5.17	64.60
Weak Scaling (4 cores/exec)	16	4	4	10.03	62.70
Weak Scaling (4 cores/exec)	32	8	4	14.94	46.70
Weak Scaling (4 cores/exec)	48	12	4	23.79	49.60
Weak Scaling (4 cores/exec)	64	16	4	34.79	54.40

6.2 Overall Summary Statistics

Table 13 provides a high-level summary of the entire experimental campaign.

Table 13: Summary Statistics of All Experiments

Metric	Value
Total Experiments	58
Strong Scaling Runs	15
Weak Scaling Runs	33
Memory Study Runs	10
Avg Execution Time (s)	1477.15
Min Execution Time (s)	316.07
Max Execution Time (s)	3021.64
Avg GC Time (s)	346.20
Avg GC Ratio (%)	1.74
Avg Compute Ratio (%)	0.95
Avg Comm Ratio (%)	2.85
Avg Peak Heap (MB)	2683.78
Total Memory Spill (GB)	689.92
Total Disk Spill (GB)	208.88
Best Configuration (Execs×Cores)	16×4
Best Time (s)	316.07
Most Efficient Config (by time/core)	32×4
Time per Core (s/core)	2.57

6.3 Best Performance Configuration

Based on the strong scaling analysis, the best performance was achieved with 16 executors each running with 4 cores (64 total cores). This configuration executed the 7.8 GB dataset in 316.07 seconds, offering the best trade-off between resource usage and execution time.

6.4 Most Efficient Configuration

The most efficient configuration, defined by the lowest time per core, was 32 executors with 4 cores each (128 total cores), with a time per core of 2.57 seconds. While this configuration was only marginally faster than the 64-core setup, it demonstrates that for very large clusters, time per core can remain low if the workload is sufficiently parallelizable.

For weak scaling, the most efficient configuration (lowest time per unit data) was 16 executors with 4 cores each (64 total cores), processing the 31 GB dataset in 1205.86 seconds, with a time per unit data of 147.82 seconds per size factor.

6.5 Configuration Recommendations

- **Executor config: 4 cores/executor, 4–5 GB memory.** Lowest coordination overhead, adequate shuffle buffers, no wasted heap above the 2.7 GB working set.
- **Core count: 32–64 total cores** for datasets ≤30 GB. Efficiency is 37–48%. Beyond 64 cores, hardware is wasted for this workload.
- **Memory: 4 GB/executor is optimal.** More memory increases execution time (YARN effects) while reducing spill by only ∼19%.

- **Partitioning:** Repartition to 200 after S1 WARC parsing. For datasets >30 GB, use `repartition(400+)`.

6.6 Algorithmic Redesign Priorities

1. **S5a Replace`approxSimilarityJoin`:** Implement custom banding that hashes MinHash signatures into band buckets and groups by bucket ID without a full self-join. Reduces shuffle from $O(n^2)$ to $O(n \cdot b)$. Expected: 5–10× speedup at large scale.

2. **S1 – Pre-split large WARC files:** Split files >1 GB into ∼200 MB segments before ingestion, enabling true file-level parallelism beyond WARC file count.

3. **S3 – Apache Arrow Pandas UDFs:** Replace `mapPartitions` with Arrow-based Pandas UDFs to eliminate JVM↔Python serialization. Expected: 2–3× speedup for the extraction stage.

7 Conclusion

This scalability study provides a detailed performance analysis of an LLM data preprocessing pipeline. The key findings are:

1. **Strong Scaling:** The pipeline scales effectively up to 64 cores for a 7.8 GB dataset. Beyond this point, communication overhead from shuffle-intensive operations (like the domain filter join and deduplication) becomes the dominant bottleneck, limiting further speedup. Strong scaling achieves 2.97× speedup (37% efficiency) at 64 cores before saturating completely due to a 67% serial fraction dominated by S5a's MinHash shuffle overhead.

2. **Weak Scaling:** The pipeline handles increasing data sizes, but execution time does not remain constant. Configurations with 4 cores per executor significantly outperform those with 2 cores, demonstrating the benefit of more powerful executors for compute-intensive workloads. Memory spill increases linearly with data size, contributing to performance degradation. Weak scaling degrades 55% over 16× data scale-up, with shuffle read time exploding to 3,450 s at 64 cores.

3. **Memory Sensitivity:** Allocating sufficient memory to executors is critical to minimize performance-degrading disk spill and GC overhead. For a 31 GB dataset with 16 two-core executors, 8 GB of memory per executor was found to be the optimal balance. Memory spill (≈11.7 GB average) is structurally driven by the LSH self-join and is completely insensitive to executor heap allocation – counter-intuitively, allocating more memory *increases* execution time.

4. **Recommendations:** For jobs similar to this pipeline, we recommend:

   - The recommended production configuration is **4 cores/executor, 4–5 GB memory, 8–16 executors** for datasets under 32 GB. Scaling beyond that requires algorithmic redesign of S5a – the pipeline's single greatest obstacle to production-scale deployment.
   - Using executors with 4 cores, as they provide a better ratio of computational power to memory.
   - Allocating memory such that intermediate data can be held in memory to avoid spill, with 8 GB per executor serving as a good baseline for similar data volumes.
   - Recognizing that the pipeline's scalability is limited by shuffle-heavy operations; future optimization efforts should focus on minimizing shuffle in stages like domain filtering and LSH-based deduplication.

Code & Scripts: https://github.com/ChethanPutran/SSDS

8 Appendix

This section provides detailed definitions for all metrics used in the scalability analysis of the PySpark LLM preprocessing pipeline. These metrics are collected during pipeline execution and used to evaluate performance, resource utilization, and scaling characteristics.

Core Configuration Metrics

Metric	Definition
Executors	Number of JVM processes launched to execute tasks. Each executor runs on a node in the cluster and manages resources for a set of tasks. Executors are launched at the beginning of a Spark application and run for its entire lifetime.
Cores per Executor	Number of CPU cores allocated to each executor. This determines how many concurrent tasks each executor can run simultaneously. Also known as executor cores or executor CPU slots.
Total Cores	Product of number of executors and cores per executor. Represents the maximum parallel capacity of the cluster: $$\text{TotalCores} = \text{Executors} \times \text{CoresPerExecutor}$$
Dataset Size	Identifier for the input data size used in the experiment. Common values: `small_1`, `small_2`, `small_4` representing relative data volumes. The numeric suffix indicates the relative size factor.
Scenario	Type of scaling experiment performed: • **Strong Scaling**: Fixed problem size, increasing resources • **Weak Scaling**: Problem size increases proportionally with resources • **Memory Study**: Fixed problem size and cores, increasing memory

Timing Metrics

Metric	Definition
App Time	Total wall-clock time from application start to completion, measured in seconds. This includes all processing stages, overhead, and cluster communication. This is the primary metric for end-to-end performance evaluation. $$\text{AppTime} = \sum_{i=1}^{n} \text{Stage}_i + \text{Overhead}$$
Compute Time	Total time spent executing actual computation tasks across all executors, measured in milliseconds. This excludes time spent on I/O, shuffling, and garbage collection. Higher compute time indicates CPU-intensive operations. $$\text{ComputeTime} = \sum_{\text{all tasks}} \text{TaskComputeDuration}$$
GC Time	Total time spent on JVM garbage collection across all executors, measured in milliseconds. High GC time indicates memory pressure and can significantly impact performance. $$\text{GCTime} = \sum_{\text{all executors}} \text{ExecutorGCTime}$$
Shuffle Read Time	Total time spent reading shuffle data from other executors, measured in milliseconds. Shuffle occurs when data needs to be redistributed across partitions (e.g., during `groupBy` or `join` operations).
Shuffle Write Time	Total time spent writing shuffle data to be consumed by other executors, measured in milliseconds. This includes serialization and disk I/O for intermediate shuffle data.
Communication Time	Total time spent on network communication between executors, measured in milliseconds. This includes shuffle read/write, data transfer, and control messages. $$\text{CommTime} = \text{ShuffleReadTime} + \text{ShuffleWriteTime} + \text{NetworkOverhead}$$

Ratio Metrics

Metric	Definition
Compute Ratio	Percentage of total execution time spent on actual computation: $$\text{ComputeRatio} = \frac{\text{ComputeTime}}{\text{TotalExecutionTime}} \times 100\%$$ Higher values (closer to 100%) indicate better efficiency as less time is spent on overhead.
GC Ratio	Percentage of total execution time spent on garbage collection: $$\text{GCRatio} = \frac{\text{GCTime}}{\text{TotalExecutionTime}} \times 100\%$$ Values above 10-15% typically indicate memory configuration issues.
Communication Ratio	Percentage of total execution time spent on network communication: $$\text{CommRatio} = \frac{\text{CommunicationTime}}{\text{TotalExecutionTime}} \times 100\%$$ High communication ratio suggests data shuffling bottlenecks.

Parallelism Metrics

Metric	Definition
Parallelism	Average number of concurrent tasks running across all executors during the application. This indicates how well the cluster resources are utilized. $$\text{Parallelism} = \frac{\sum_{\text{time window}} \text{ConcurrentTasks}}{\text{TimeWindow}}$$
Utilization	Percentage of theoretical maximum parallelism actually achieved: $$\text{Utilization} = \frac{\text{Parallelism}}{\text{TotalCores}} \times 100\%$$ Values close to 100% indicate excellent resource utilization.

Memory Metrics

Metric	Definition
Peak Heap Memory	Maximum JVM heap memory usage across all executors during application execution, measured in MB. This indicates the memory pressure and helps in sizing executor memory. $$\text{PeakHeap} = \max_{\text{all executors}} (\text{ExecutorHeapUsage})$$
Memory Spill	Amount of data spilled from memory to disk due to insufficient RAM, measured in MB. Spilling occurs when operations (like shuffles or aggregations) exceed available memory. High spill values indicate memory configuration is too small.
Disk Spill	Amount of data written directly to disk (bypassing memory) during shuffle operations, measured in MB. This is more severe than memory spill and significantly degrades performance.

Scaling Metrics

Metric	Definition
Speedup	Factor by which execution time decreases as resources increase: $$\text{Speedup} = \frac{\text{Time}_{\text{baseline}}}{\text{Time}_{\text{current}}}$$ where baseline is typically the smallest configuration.
Ideal Speedup	Theoretical maximum speedup with perfect parallelization: $$\text{IdealSpeedup} = \frac{\text{TotalCores}_{\text{current}}}{\text{TotalCores}_{\text{baseline}}}$$
Efficiency	How close actual speedup is to ideal speedup: $$\text{Efficiency} = \frac{\text{Speedup}}{\text{IdealSpeedup}} \times 100\%$$

Pipeline Stage Metrics

Metric	Definition
S1 - WARC to Parquet	Time to parse raw WARC files, extract HTML content, and write to Parquet format (Step 1).
S2 - Domain Filtering	Time to load blacklist, extract domains from URLs, and filter out blacklisted domains (Step 2).
S3 - Text Extraction	Time to extract clean text from HTML content (Step 3).
S4 - Language Filtering	Time to detect language and filter non-English content (Step 4).
S5a - Deduplication (MinHash)	Time to compute MinHash signatures and find near-duplicate documents (Step 5a).
S5b & Exact Deduplication (MD5)	Time to perform connected components clustering on similar documents (Step 5b).
S6a - Tokenization	Time to tokenize text and prepare for model input (Step 6a).
S6b - Final Write	Time to write processed data to final output format (Step 6b).

Derived Metrics for Analysis

Metric	Definition
Time per Core	Execution time normalized by total cores. Useful for comparing efficiency across configurations: $$\text{TimePerCore} = \frac{\text{AppTime}}{\text{TotalCores}}$$
Time per Unit Data	Execution time normalized by dataset size (for weak scaling analysis): $$\text{TimePerUnitData} = \frac{\text{AppTime}}{\text{DatasetSizeFactor}}$$
Stage Contribution	Percentage contribution of each stage to total execution time: $$\text{StageContribution}_i = \frac{\text{StageTime}_i}{\text{AppTime}} \times 100\%$$